
Uncovering Intra-expert Activation Sparsity for Efficient Mixture-of-Expert Model Execution

Jongseok Park*

University of California, Berkeley
js_park@berkeley.edu

Sunga Kim

University of California, Berkeley
sunga.kim@berkeley.edu

Zhenyu Gu

Advanced Micro Devices, Inc.
Zhenyu.Gu@amd.com

Ion Stoica

University of California, Berkeley
istoica@berkeley.edu

Alvin Cheung

University of California, Berkeley
akcheung@berkeley.edu

Abstract

Mixture of Experts (MoE) architecture has become the standard for state-of-the-art large language models, owing to its computational efficiency through sparse expert activation. However, sparsity through finer expert granularity is becoming increasingly difficult to achieve due to fundamental training challenges such as expert collapse and load imbalance. In this work, we explore and leverage **intra-expert activation sparsity** as a complementary and underexplored dimension of sparsity in MoE models. Surprisingly, substantial intra-expert sparsity is *readily available in existing pre-trained MoE models*, without any modification to the activation function or model parameters, providing up to **90% sparsity** within each expert without significant accuracy loss. We explore intra-expert activation sparsity across eight off-the-shelf MoE models ranging from 1B to 400B parameters, and extend the MoE execution pipeline of vLLM to leverage intra-expert activation sparsity by skipping the computations of inactive neurons, on top of its existing optimizations, achieving up to $2.5\times$ speedup in MoE layer execution and $1.2\times$ end-to-end speedup compared to the original dense vLLM baseline.

1 Introduction

Mixture of Experts (MoE) architecture [41, 10] has become the de facto standard for state-of-the-art Large Language Models (LLMs) [38, 36, 31], owing to its superior computational efficiency over traditional dense LLM architectures. MoE models use a sparse feed-forward network (FFN) composed of multiple independent sub-networks called *experts*. Each expert typically specializes in a distinct domain of knowledge [51], and during inference, the model selectively activates only those experts most relevant to the current input token. Unlike dense models, which activate all parameters, this sparse activation allows MoE models to reduce computation while maintaining high model capacity. As a result, MoE models have evolved to increase the number of experts and maximize sparsity by reducing the percentage of active experts.

Training these sparse MoE models is notoriously difficult. Without careful regularization, MoE models are prone to expert collapse [10, 56], where the model converges to using a few general

*Work done during an internship at Advanced Micro Devices, Inc.

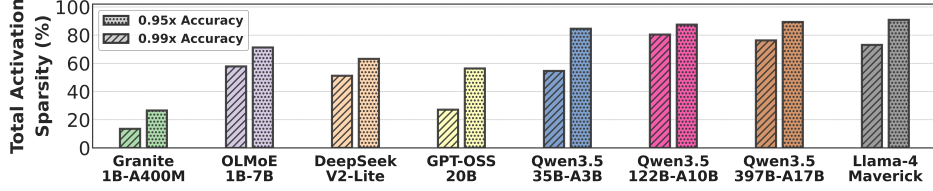


Figure 1: Maximum total intra-expert activation sparsity that retains 95% and 99% of the average benchmark score of the unmodified execution. Benchmark details are available in 3.1.

experts while leaving the majority of experts untrained, or load imbalance [10, 55], where hotspots in expert activation degrade both model quality and hardware utilization. Additionally, MoE models suffer from representation collapse where experts converge to similar specializations [6] and reduced per-expert gradient updates [37]. These challenges only compound as the number of experts grows, posing fundamental difficulties for further increasing *inter-expert sparsity*.

Instead of pushing *inter-expert sparsity* further, we explore sparsity within the activations of each expert, or **intra-expert activation sparsity** as a complementary source of sparsity in MoE models at inference time. Activation sparsity has been widely studied in model architectures with ReLU activations that explicitly zero negative activations [14], but recent studies reveal that activation sparsity exists in newer LLM architectures as well [43, 5, 21, 25], despite their use of leaky activation functions such as SwiGLU [40]. Activation outputs of LLM models are known to follow a Sparsing Law [28], where the fraction of important neurons decreases as the model parameter size grows. For example, activation outputs of models such as Llama-2 [45] that use SwiGLU follow a long-tailed distribution with longer tails in the larger variants of their model family [43]. The same phenomenon has also been validated on MoE models [5] such as Qwen3 [47] and Mixtral [17], confirming that activation sparsity is not an optimization axis restricted only to ReLU-based LLMs.

However, we find **two major obstacles** that inhibit the widespread use of intra-expert activation sparsity for efficient MoE inference. First, many existing works target older dense architectures or require training to achieve sparsity. This greatly limits the scope and practicality of the technique, calling into question whether the approach is applicable to more widely used *pretrained MoE models*. Secondly, there is no *end-to-end integration* of intra-expert activation sparsity in highly optimized execution systems. Production systems such as vLLM [19] include sophisticated MoE kernels and techniques such as expert parallelism [8, 22] to maximize performance. However, existing works only implement a naive research-level baseline, leaving a critical gap in evaluating the compatibility and benefit of intra-expert activation sparsity in high-performance systems.

To this end, we first conduct a thorough study of intra-expert activation sparsity on off-the-shelf MoE models ranging from 1B to 400B parameters and report a surprising finding: substantial intra-expert activation sparsity is *readily available* in many existing MoE models, without requiring any modification to the model. By ranking neurons by their activation scores and deactivating the lowest-scoring ones, we measure up to **90% sparsity** within active experts while retaining 95% of the original model accuracy, as shown in Figure 1. We also find that neurons from shared and routed experts have markedly different impacts on accuracy, to the point when all shared expert neurons are active, as few as 2% of the active routed expert parameters suffice to maintain model capacity for large models such as Qwen3.5-122B-A10B.

Based on this study, we extend the fused MoE execution pipeline of the popular production-grade inference system vLLM to leverage intra-expert activation sparsity by skipping the up and down projection computations for inactive neurons. Our implementation builds on top of the expert-batched MoE kernel design of vLLM, retaining **full support for existing optimizations** such as HIP Graph [3] and CUDA Graph [35], kernel fusion [2], and expert parallelism. We also share the insights we gained during our implementation to clearly illustrate the benefits and limitations of intra-expert sparsity in an end-to-end production setting.

In summary, we make the following contributions. (1) We explore intra-expert activation sparsity across eight off-the-shelf MoE models, with thorough analysis on their activation sparsity characteristics. (2) We redesign the MoE execution pipeline of vLLM to efficiently exploit both *inter-* and *intra-expert sparsity* on GPUs. (3) We integrate our sparse execution pipeline into the full vLLM engine and demonstrate performance gains, achieving up to $2.5\times$ speedup in MoE layer execution and $1.2\times$ speedup in end-to-end inference. Together, these contributions establish intra-expert activation

"Experts" and "Active Experts" refer to the number of experts and active experts *excluding* shared experts.

Model Name	Params (Active / Total)	Experts	Active Experts	FFN Dim.	Shared Experts
Granite-1B-A400M [15]	400M / 1B	32	8	512	False
OLMoE-1B-7B [33]	1B / 7B	64	8	1024	False
DeepSeek-V2-Lite [8]	2.4B / 16B	64	6	1408	True
GPT-OSS-20B [36]	3B / 20B	32	4	2880	False
Qwen3.5-35B-A3B [38]	3B / 35B	256	8	512	True
Qwen3.5-122B-A10B [38]	10B / 122B	256	8	1024	True
Qwen3.5-397B-A17B [38]	17B / 397B	512	10	1024	True
Llama-4-Maverick [31]	17B / 400B	128	1	8192	True

Table 1: Architectural details of the tested MoE models

sparsity as a practical optimization technique for MoE model execution and lay the groundwork for future efficient LLM research.

2 Background and Related Work

Feed-Forward Network: The feed-forward network (FFN) of the Transformer architecture [46] applies a non-linear activation function in a higher-dimensional latent space to encode and process factual and linguistic knowledge [13], and often dominates the parameter count of LLMs. Each dimension of the intermediate latent space of an FFN corresponds to an individual neuron whose contribution to the output is weighted by its activation value. Early work has shown that FFN neurons specialize in distinct aspects of the input, and a large fraction of neurons return near-zero activation values, effectively becoming inactive for that input [51, 32].

Mixture of Experts: Mixture of Experts (MoE) FFN architecture leverages the specialization of neurons by replacing the single FFN with multiple independent expert FFNs, where each expert learns to specialize on a distinct subset of inputs [41, 10]. During inference, a router network determines which experts should be activated for a given input token, and the vast majority of FFN neurons belonging to unrelated experts are efficiently bypassed, providing coarse-grained *inter-expert sparsity*. However, training such MoE models is known to be extremely difficult [10, 56, 6], as the model must simultaneously specialize each expert toward distinct domains while ensuring equal importance of the domains for balanced expert utilization. Since expert specialization must also emerge *implicitly* during training rather than being explicitly prescribed, MoE training is highly unstable without careful regularization, making models prone to expert collapse [56] or load imbalance [55].

Activation Sparsity: Sparsity in neural networks can also be achieved through activation sparsity. Unlike inter-expert sparsity, which is built into the MoE model architecture, activation sparsity exploits the tendency of *activation functions* to suppress a large fraction of neuron outputs for any given input. For example, the hard zero threshold of ReLU activation causes a significant fraction of neurons to be completely inactive [14], allowing computations to be skipped entirely. While activation sparsity was pioneered in the context of convolutional neural networks (CNNs) that used ReLU extensively [18, 42], works such as DeJaVu [26] demonstrated the potential of activation sparsity for efficient LLM inference using ReLU-activated LLMs, such as OPT [50], and have shown over 90% sparsity [32] in its FFN layers.

Recent studies confirm the existence of activation sparsity in models with leaky GeLU [16] or SiLU [9] based Gated Linear Unit (GLU) activations such as GEGLU or SwiGLU [40], despite their lack of exact zeros [43, 5, 21, 25]. Sparsing Law [28] establishes that the fraction of activated neurons decreases as the model parameter size increases, regardless of the activation used. This observation is empirically shown to be true in LLMs using leaky activations [43], such as Gemma3 [12] and Qwen 2.5 [48], and also MoE models such as Qwen3-30B-A3B [47] and Mixtral-8x7B [17]. These works collectively establish that while leaky activations lack exact zeros, many FFN neurons remain unimportant and can be leveraged for activation sparsity through appropriate thresholding.

3 Intra-expert Activation Sparsity

3.1 Methodology

We hypothesize that when a token is routed to an expert, it only needs to activate a small subset of that expert’s neurons, not the expert as a whole. We refer to this as **intra-expert activation sparsity**. To

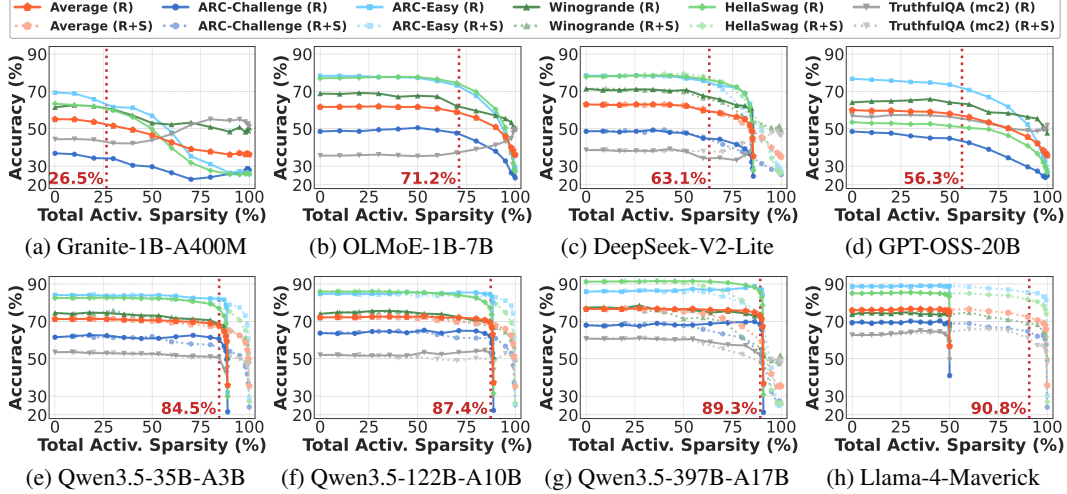


Figure 2: Accuracy benchmark of MoE models with varying levels of total activation sparsity. Sparsity is applied only to routed experts in R results, and sparsity is applied to both routed and shared experts in R+S results. Sparsity cutoff of the model is annotated with a vertical red dotted line.

validate this claim, we take pre-trained MoE LLMs and measure their benchmark scores at different levels of intra-expert activation sparsity, by sorting each expert’s post-activation outputs and zeroing out the lowest-scoring neurons. We do not modify any other part of the model architecture or update the model parameters in any way.

We test eight MoE models as shown in Table 1 against five benchmarks, namely ARC-Challenge [7], ARC-Easy [7], HellaSwag [49], Winogrande [39], and TruthfulQA-mc2 [24], to cover a wide variety of models and benchmarks. We use lm-eval-harness [11] with vLLM [19] backend, and evaluate both with and without applying intra-expert sparsity to the shared experts for models with shared experts. We define the maximum sparsity at which the model retains 95% of its baseline average benchmark score as the representative *sparsity cutoff* value of the model. License details of the assets are available in Appendix A.

3.2 Observation and Analysis

We first present an overview and take a deeper dive comparing the importance of (1) neurons among the shared experts, (2) neurons across different routed experts, and (3) neurons within a single expert, to find which neurons are most influential for accuracy and guide the application of activation sparsity.

Figure 2 presents the accuracy benchmark of the tested models. As noted in previous studies, we observe the trend that larger models generally have increased robustness to activation sparsity, with the smallest Granite-1B-A400M having the lowest sparsity cutoff at 26.5% and the largest Llama-4-Maverick having the highest cutoff at 90.8%. However, unlike previous studies that only covered models up to 30B parameters [43, 5], our experiment extends the model size up to 400B and confirms that intra-expert activation sparsity scales well beyond models with 100B parameters with even higher sparsity levels. From this observation, we conclude that intra-expert activation sparsity is not only readily available in existing models, but also **increasingly useful** in larger frontier models.

Shared Expert Neurons: We find that shared experts have higher per-neuron impact on accuracy compared to neurons of routed experts. For example, in Figure 2c, DeepSeek-V2-Lite is shown to retain its accuracy over a wider range of sparsity in its routed expert-only (R) results compared to GPT-OSS-20B in Figure 2d, despite the smaller parameter count of 16 billion. However, in the dotted (R+S) results where the sparsity is also applied to the shared experts, its accuracy curve becomes similar to GPT-OSS-20B with a wider arc and reduced accuracy retention. Similar trends are also observed in other models with shared experts, where the (R) results show a much sharper knee and constantly higher accuracy against the (R+S) results across all sparsity.

This phenomenon is more pronounced in larger parameter models, to the point where **less than 2% of the active routed neurons is enough to retain model capacity** for each token when combined with the full shared expert, as shown by the sparsity cutoff of the Qwen3.5-122B-A10B

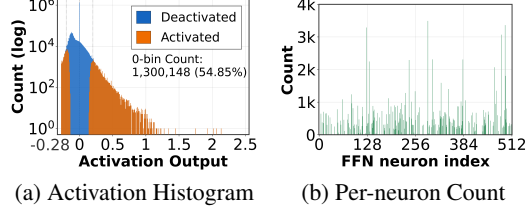
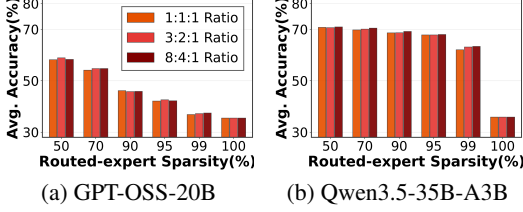


Figure 3: Average accuracy when different ratio of neurons are allocated per expert based on router weight. Legend is shared between subfigures.

Figure 4: Activation output histogram and per-neuron activation count of expert 0, layer 0 of Qwen3.5-35B-A3B at 95% activation sparsity.

and Qwen3.5-397B-A17B models. While prior work [38, 8] has pursued higher sparsity through inter-expert sparsity using finer expert granularity, achieving an equivalent 2% activation level via inter-expert sparsity alone would require scaling the total number of experts by approximately $50\times$, which would significantly worsen the training challenges such as expert collapse and load imbalance discussed in Section 2.

Neurons across Routed Experts: To check if differences in neuron importance also exist within different routed experts, we distribute a *neuron budget* across active routed experts. We sort and divide the activated experts into three groups g_0, g_1, g_2 based on their router weights (descending), with the remaining experts assigned to g_2 if the number of active experts K is not divisible by three. Given a sparsity target s , we first calculate the total neuron budget $N = s \times K \times D$, where D is the FFN dimension of the model. Then, based on the distribution ratio r_x , we allocate $N_{g_x} = \frac{N \times r_x}{\sum r_x \times |g_x|}$ neurons to the experts of the group g_x , where $|g_x|$ is the number of experts in the group g_x . Figure 3 shows the resulting average accuracy under three different ratio configurations, shown in $r_0 : r_1 : r_2$ order. It is clear that, while allocating more neurons to experts with higher router priority does increase accuracy, **the benefits are minimal** with only up to 2 percentage points improvement.

This can be interpreted as a fine-grained analogue of expert skipping [27, 54], which skips entire experts with low router scores for efficient MoE inference. While further studies on a more sophisticated neuron budgeting algorithm, such as router-weight-based waterfilling, may increase its benefits, we decide it is outside the scope of this paper. Therefore, we do not utilize neuron budgeting in our end-to-end system and leave it for future optimizations of intra-expert activation sparsity.

Neurons within a Single Expert: To compare the impact of neurons within a single expert, we profile the SwiGLU activation output of expert 0, layer 0 of Qwen3.5-35B-A3B at 95% sparsity on 2048 samples of the WikiText-2 dataset. Figure 4a presents the histogram of the activation outputs before and after masking the bottom 95% activations, and Figure 4b presents the activation count of each neuron after masking. Note that the y-axis of Figure 4a is on a logarithmic scale.

From Figure 4a, it is clear that the activation output of MoE expert FFN follows a long-tailed distribution, with an exponential decrease in count as the output magnitude increases. We also note the extreme spike in the 0-bin that includes the activations between -0.003 to 0.003, where **54.85% of the total activation output is zero or near zero**. As such, even at 95% sparsity, only the neurons with activation output between -0.2 to 0.2 are deactivated (blue), while the much larger range of activation output stays activated (orange).

As seen from Figure 4b, expert activation at 95% sparsity is also extremely uneven across neurons with fewer than 10 neurons receiving more than 2000 activations, which is $8.5\times$ the average of 235.12 activations. In addition, 284 neurons are never activated, roughly matching the 55% observed in the 0-bin in Figure 4a. This observation shows that the neurons of MoE FFN layer are highly parameter inefficient and confirms the opportunity not only for intra-expert activation sparsity to reduce computation, but also for neuron pruning [20, 29] to reduce model size as well.

4 Evaluation

4.1 Intra-expert Activation Sparse MoE in vLLM

Based on these observations, we integrate intra-expert activation sparsity in vLLM as a *separate activation-sparse execution codepath* in the routed expert execution code, while leaving other parts

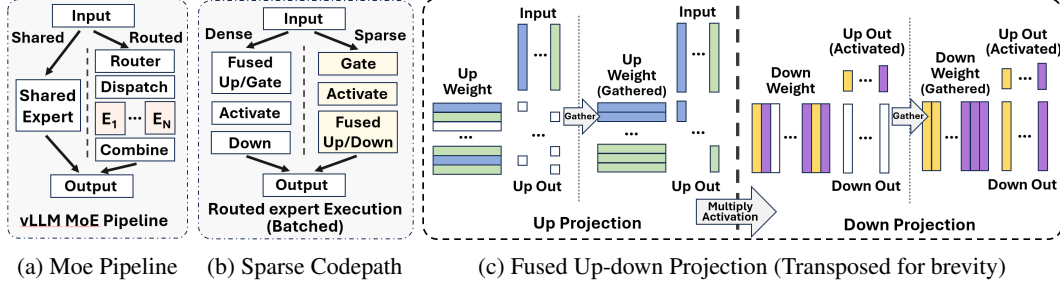


Figure 5: Overview of intra-expert activation sparsity integration in vLLM MoE execution pipeline.

of the system intact for compatibility and performance. The sparse execution path is activated when the user provides a **target activation sparsity** as an argument to the vLLM engine at launch. Full source code of our implementation is available in Appendix B and supplementary materials.

As shown in the vLLM MoE Pipeline of Figure 5a, we exclusively modify the routed per-expert execution code highlighted in orange without changing shared expert execution or the router, dispatch, and combine functions that surround the per-expert execution. Applying activation sparsity to the shared expert has little to no gain, as they are executed in parallel to the routed expert path in the vLLM engine, and their neurons have a high impact on accuracy. Similarly, router, dispatch, and combine functions in routed execution are critical to the performance of expert parallelism (EP) and often use hand-crafted all-to-all communication libraries such as RCCL [1], NCCL [34], or DeepEP [53], which cannot be easily modified without severely impacting parallelism performance.

Figure 5b visualizes the changes we made to the routed expert execution. We add the activation sparse execution path on the right, highlighted in yellow. The execution path can be dynamically switched between the original dense expert execution of vLLM and our sparse execution path, depending on the step batch size, as explained later. Note that the operators such as up, gate, activate, and down projections are batched across the entire routed experts E_1 to E_N in the given EP rank for efficiency in both dense vLLM and our sparse execution path, which is skipped in the figure for brevity. Our sparse code has **three key characteristics** that differ from the dense execution: (1) Standalone gate projection, (2) Custom sparse activation kernel, and (3) Fused sparse up-down projection kernel.

Gate projection must be executed before all sparse operations, as its output is necessary to compute the neuron activations and sparsity mask. To efficiently compute gate projection, we reuse the *batched matrix multiplication* (BMM) kernel used for the down projection of the dense path, which provides high-throughput expert-major computation and can be easily repurposed by providing the gate weights as input. As the gate projection uses a dense BMM kernel, it limits the scope of sparse computation to roughly two-thirds of the execution (up and down), and maximum speedup to $3\times$. However, we find that overall execution still yields substantial performance improvements, as shown in Section 4.2.1. Nevertheless, further studies on fast approximation or activation sparsity mask reuse, similar to those in sparse KV cache [52, 23], may allow for a fully sparse expert execution and enable higher performance in intra-expert activation sparsity.

We implement a custom sparse activation kernel that fuses the activation calculation and masking of the deactivated neurons into a single kernel using Triton [44]. The kernel computes neuron activations using the gate projection output and applies threshold-based single-pass masking [25, 21]. The user-supplied sparsity argument is turned into a model-specific threshold using a lookup table generated during engine startup. If a neuron’s absolute activation output value is below this threshold, the neuron is masked out. This allows for an $\mathcal{O}(n)$ sparsity mask generation compared to the traditional $\mathcal{O}(n \log n)$ top-k approach, with minimal accuracy loss as shown in Section 4.2.2. The activation kernel then gathers the activated neuron indices using the mask and returns it in a continuous buffer.

Finally, we create a fused sparse up-down projection kernel based on the Triton BMM kernel provided by vLLM. As shown in Figure 5c, we first use batched vector load to efficiently *gather the up weight vectors* of each activated neuron for a given input token, using the active neuron index buffer generated by the activation kernel. This creates a packed, dense up weight matrix on the shared memory of the GPU, which can be efficiently computed using a dense matrix-vector multiplication against the input token to generate the up projection output. The output is multiplied by the activation and used as an input to the matrix-vector multiplication with the down-projection weights, which are also gathered and packed into a dense matrix using batched vector loads.

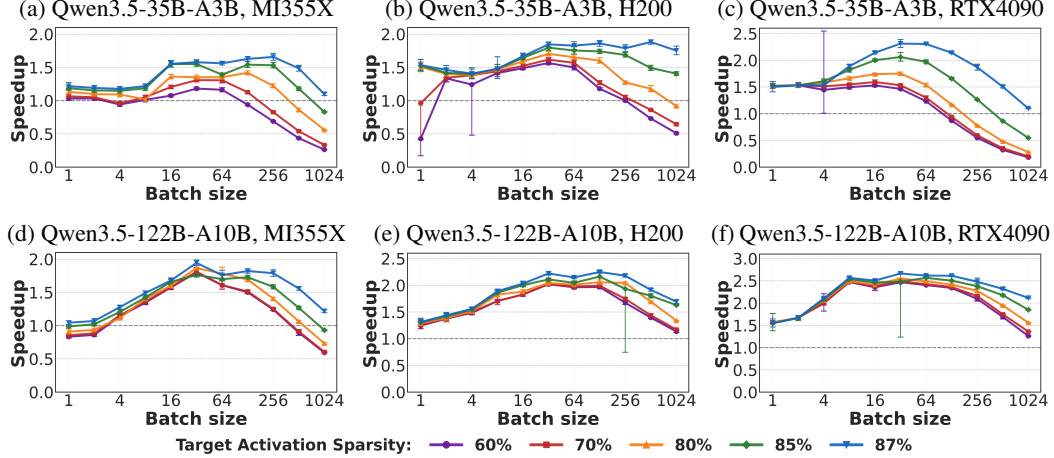


Figure 6: MoE layer speedup of intra-expert activation sparse execution against dense vLLM on various settings, averaged across 100 runs with a 96% confidence interval. Note that Qwen3.5-35B-A3B and Qwen3.5-122B-A10B have a sparsity cutoff of **84.5%** and **87.4%**, respectively.

To maintain compatibility with Hip Graph and CUDA Graph, which require reuse of the same GPU memory buffers and kernel launch parameters across graph invocations, the gathered active neuron index buffer is padded into the full neuron size, and the Triton kernels are launched with a tile granularity of a fixed 64 neurons per Triton program. Each Triton program first checks its assigned neuron tile from the active neuron index vector, and exits early if its assigned tile only contains padding indices. This allows for efficient implementation of sparse kernel execution while keeping compatibility with graph capture optimizations, and the fixed 64 neuron tile granularity enables utilization of GPU hardware accelerators for batched memory access and high-throughput computation instructions.

This approach has a limited maximum throughput compared to the dense BMM in the compute-bound region with large batch sizes. This is a fundamental limitation of a gather-based sparse execution solution, as GPU hardware provides the highest throughput when executing dense operations with contiguous memory access. To overcome this limitation, we profile the tipping batch size between the sparse execution path and dense execution path during engine startup. Based on this, we switch between sparse and dense execution paths based on the batch size of the running step. This allows us to use the low-latency sparse path when there are fewer workloads, and switch back to the high-throughput dense path when there are more workloads.

4.2 Performance Evaluation

4.2.1 MoE Layer Performance

We integrate our intra-expert activation sparse MoE execution on a development build of vLLM [19]², and evaluate its benefits by comparing its speedup against the original dense vLLM baseline. We use AMD MI355X, NVIDIA H200, and NVIDIA RTX4090 GPUs for our experiments. We use Qwen3.5-35B-A3B and Qwen3.5-122B-A10B models [47], with samples from WikiText-2 [30] dataset as input to our experiments. We use the total activation sparsity levels of 60%, 70%, 80%, 85%, and 87%, covering a representative set of the model sparsity cutoff values observed in Section 3.

Figure 6 presents the speedup of intra-expert activation sparse MoE execution over the dense vLLM baseline, measured while executing the first MoE layer of the given model. The sparse execution achieves up to $1.5\text{--}2.5\times$ speedup at the 85% and 87% sparsity cutoff for Qwen3.5-35B-A3B and Qwen3.5-122B-A10B. As expected, the speedup is highly dependent on batch size, with peak speedup observed between batch sizes of 16 to 128. At smaller batch sizes, static overhead such as kernel launch cost dominates, offsetting the computation savings of sparse execution and giving flat speedup. At larger batch sizes, execution approaches the compute-bound regime, where the higher peak

²Version 0.1.dev15965+g6506aec54

Target Actv. Sparsity	Total Actv. Sparsity	Routed Expt. Sparsity	Average Accuracy	Time Taken	Speedup
Dense	-	-	70.9%	0.354ms	-
60%	57%	64%	68.2%	0.383ms	0.92×
70%	72%	81%	67.6%	0.315ms	1.12×
80%	79%	89%	67.7%	0.249ms	1.42×
85%	84%	94%	67.5%	0.229ms	1.55×
87%	87%	98%	65.7%	0.197ms	1.8×

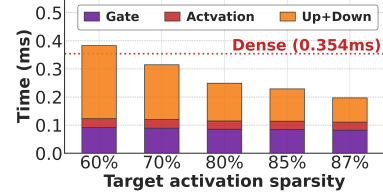


Table 2: Qwen3.5-35B-A3B achieved total activation and routed expert sparsity, average benchmark accuracy, and time taken on various target activation sparsity on MI355X, Batch size=128

Figure 7: Qwen3.5-35B-A3B MoE layer sparse execution time breakdown on MI355X, Batch size=128

throughput of the dense execution outweighs the benefits of sparsity. Together, these two effects cause the speedup curve to form a convex shape across batch sizes.

Sparsity plays a key role in the speedup, as both the peak speedup and the range of batch sizes with speedup scale with the degree of sparsity. To be specific, at smaller batch sizes the speedup differences across sparsity levels are less pronounced, as the absolute reduction in computation is small and static overheads such as the dense gate projection and activation kernel dominate. As batch size increases and execution approaches the compute-bound regime, the degree of sparsity plays a more decisive role: lower sparsity levels of 60–70% see a sharper decline in speedup, while higher sparsity levels of 85–87% remain more robust across a wider batch size range.

Hardware and model size also greatly affect the efficacy of sparse execution. The general trend is that **sparse execution provides greater speedup when resource bottlenecks are more severe**. Comparing the MI355X, H200, and RTX4090—where the MI355X has the most raw compute and memory resources and the RTX4090 has the least—we find that the RTX4090 achieves the highest speedup of up to $2.5\times$, while the MI355X reaches only up to $1.8\times$, with the H200 in between at $2.0\times$. Similarly, we observe roughly 30–40 percentage points increases in speedup across all configurations on Qwen3.5-122B-A10B compared to Qwen3.5-35B-A3B, confirming the greater benefits of sparse execution on more compute- and memory-intensive workloads. As such, the highest speedup of $2.5\times$ is observed while running the larger Qwen3.5-122B-A10B MoE layer on the resource-constrained RTX4090 GPU.

4.2.2 Sparsity, Accuracy, and Execution Breakdown

Table 2 presents the achieved sparsity and end-to-end average benchmark accuracy of our threshold-based activation sparse MoE execution, running the same set of benchmarks as in Section 3.2. As described in Section 4.1, we use a lookup table to convert the user-specified target activation sparsity to a per-neuron deactivation threshold. This threshold-based masking is applied to routed experts, yielding a model-specific routed expert sparsity that translates to the desired total activation sparsity when shared experts are accounted for.

As shown in the Total Activ. Sparsity column of the table, our threshold-based approach achieves total activation sparsity within ± 3 percentage points of the desired target sparsity. Furthermore, the Average Accuracy column shows that the sparse execution maintains 95% of the baseline average benchmark score up to the target sparsity of 85%. This closely matches the model’s sparsity cutoff identified in our earlier analysis and confirms that **our sparse execution correctly reproduces the activation sparse model accuracy** observed in Section 3.2.

Figure 7 shows the time breakdown of the sparse MoE execution across different target activation sparsity levels. We see that the dense gate projection and activation kernel contribute a constant overhead to execution, and only the gather-based fused up-down projection kernel benefits from increased sparsity. As noted previously, this is a limitation of our current approach: earlier-stage sparsity map generation or reuse would allow sparsity to be applied to the gate projection as well, enabling further performance gains. Nevertheless, our threshold-based sparse execution still achieves $1.55\times$ speedup at the model sparsity cutoff, as shown in Table 2, demonstrating substantial benefits to inference performance.

GPU	Input Tokens	Output Tokens	Number of Requests						Average Speedup
			1	4	16	64	256	512	
MI355X	1024	1	1.01×	1.01×	1.01×	1.01×	1.00×	1.00×	1.01×
	1024	128	1.10×	1.08×	1.10×	1.06×	1.03×	1.00×	1.06×
	1	1024	1.04×	1.03×	1.05×	1.06×	1.07×	1.00×	1.04×
	128	1024	1.07×	1.07×	1.08×	1.07×	1.10×	1.00×	1.08×
	1024	1024	1.09×	1.09×	1.09×	1.07×	1.05×	1.00×	1.07×
H200	1024	1	1.11×	1.09×	1.08×	1.09×	1.08×	1.10×	1.09×
	1024	128	1.17×	1.15×	1.13×	1.08×	1.06×	1.06×	1.11×
	1	1024	1.18×	1.19×	1.19×	1.19×	1.19×	1.13×	1.18×
	128	1024	1.19×	1.18×	1.19×	1.19×	1.17×	1.13×	1.18×
	1024	1024	1.20×	1.20×	1.20×	1.19×	1.14×	1.13×	1.18×

Table 3: End-to-end Qwen3.5-35B-A3B inference speedup of activation sparse MoE execution at 85% target sparsity against dense vLLM baseline.

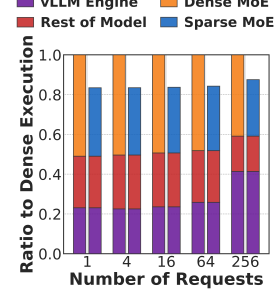


Figure 8: End-to-end execution time ratio.

4.3 End-to-end Evaluation

Table 3 presents the end-to-end model inference speedup of intra-expert activation sparse MoE execution over the dense vLLM baseline. Note that batch-size-based switching between dense and sparse MoE execution is enabled in the evaluation, guaranteeing speedup across all configurations. With the switch enabled, we observe up to $1.2\times$ speedup on H200 and up to $1.1\times$ on MI355X, with a minimum speedup of $1.0\times$ as expected. Sparse execution primarily benefits latency-bound workloads, such as decode-heavy configurations with large numbers of output tokens or small numbers of requests. In compute-bound scenarios such as prefill-heavy workloads or executions with large numbers of requests, the benefits are reduced, as the per-step batch size frequently exceeds the switching threshold and the dense path is utilized more often.

Figure 8 shows the execution time ratio of each end-to-end operation compared to the full dense vLLM execution, on H200 for end-to-end Qwen3.5-35B-A3B inference on requests with 1024 input and output tokens. “Dense MoE” and “Sparse MoE” refer to routed expert execution time measured in Section 4.2.1. “Rest of Model” is the remaining model components such as the attention or embedding layers. “vLLM Engine” refers to the serving overhead of vLLM, including request scheduling, KV cache management, and output sampling.

Across batch sizes, dense MoE layer execution time accounts for roughly 45% of total execution time. While we achieve a $1.5\text{--}1.8\times$ speedup in the MoE layer, Amdahl’s law [4] limits the end-to-end improvement to roughly a 15–20% reduction in total execution time, which corresponds to the $1.2\times$ end-to-end speedup observed in Table 3. Nevertheless, the MoE layer constitutes the majority share of model execution time across all batch sizes, making it a high-impact target for optimization.

5 Discussion

Sparse Gate Projection: The current design applies activation sparsity only on the up and down projections, which account for roughly two-thirds of the expert FFN computation. This imposes a theoretical upper bound of $3\times$ on end-to-end speedup even under optimal up/down execution. Thus, future work could explore ways to allow a lightweight gate-activation prediction mechanism that estimates sparse activation patterns before computing the full gate projection, or reusing the sparsity pattern based on the input token. Achieving accurate prediction of the sparsity map before gate projection would allow the system to prevent full dense gate computation, reducing activation overhead and enabling performance gains closer to the theoretical upper bound of intra-expert sparsity, enabling higher end-to-end MoE speedups.

Grouped Intra-expert Activation Sparsity: Our results reveal that sparsity alone does not guarantee speedup, as mapping sparse computation efficiently onto GPU hardware introduces overhead due to irregular memory access, indexing, and limited hardware utilization. This gap between theoretical sparsity and realized speedup widens as batch size increases, underscoring the need for more optimized sparse kernel designs. The most promising direction is to increase the granularity of sparsity mapping: rather than assigning and gathering each individual active neuron, tokens and neurons can be grouped to fit the hardware accelerator width and perform sparsity mapping at this block level. Minimal group sizes between 4 and 32 would better align sparse computation with the regular tile structures expected by high-performance kernels, with only a minor tradeoff in precision, leading to improved memory coalescing and increased hardware utilization.

6 Conclusion

In this work, we demonstrated how intra-expert activation sparsity can improve the efficiency of MoE models. While conventional MoE architectures reduce computation by limiting the number of activated experts per token, our approach further exploits sparsity in the dense FFN of each selected expert. We identified substantial redundancy within the dense expert itself and demonstrated that MoE inference can be further accelerated at inference time. Intra-expert activation sparsity is a promising and practical complement to inter-expert sparsity for reducing the serving cost of large MoE models, with growing relevance as model scale and inference demand continue to rise.

References

- [1] AMD. RCCL: ROCm collective communications library. <https://github.com/ROCmSoftwarePlatform/rccl>, 2020. Software library.
- [2] AMD. Fusion API: Getting started. https://rocm.docs.amd.com/projects/MIOpen/en/latest/Getting_Started_FusionAPI.html, 2024. MIOpen ROCm Documentation.
- [3] AMD. HIP graph API tutorial. https://rocm.docs.amd.com/projects/HIP/en/latest/tutorial/graph_api.html, 2024. ROCm HIP Documentation.
- [4] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS ’67 (Spring), pages 483–485. ACM, 1967.
- [5] Runxi Cheng, Yuchen Guan, Yucheng Ding, Qingguo Hu, Yongxian Wei, Chun Yuan, Yelong Shen, Weizhu Chen, and Yeyun Gong. Mixture of neuron experts. *arXiv preprint arXiv:2510.05781*, 2025.
- [6] Zewen Chi, Li Dong, Shaohan Huang, Damai Dai, Shuming Ma, Barun Patra, Saksham Singhal, Payal Bajaj, Xia Song, Xian-Ling Mao, Heyan Huang, and Furu Wei. On the representation collapse of sparse mixture of experts. In *Advances in Neural Information Processing Systems*, volume 35, pages 34600–34613, 2022.
- [7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge, 2018.
- [8] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.
- [9] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, 2017.
- [10] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [11] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024.
- [12] Gemma Team. Gemma 3 technical report, 2025.
- [13] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, Online and Punta Cana, Dominican Republic, 2021. Association for Computational Linguistics.
- [14] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15, pages 315–323. PMLR, 2011.

- [15] Granite Team, IBM. Granite 3.0 Language Models. Technical report, IBM, 2024.
- [16] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016.
- [17] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Th  ophile Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mixtral of experts, 2024.
- [18] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.
- [20] Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In *Advances in Neural Information Processing Systems*, volume 2, 1989.
- [21] Je-Yong Lee, Donghyun Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-aware thresholding for sparsity in large language models. In *Conference on Language Modeling*, 2024.
- [22] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding, 2021.
- [23] Yuhong Li, Yingbing Huang, Bowen Yang, Bhargav Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deyu Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [24] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods, 2021.
- [25] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-free activation sparsity in large language models, 2024.
- [26] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja vu: Contextual sparsity for efficient LLMs at inference time, 2023.
- [27] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [28] Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liqun Deng, Jiansheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing law: Towards large language models with greater activation sparsity, 2024.
- [29] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the structural pruning of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 21702–21720, 2023.
- [30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [31] Meta Llama Team. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. 2025. Meta AI Blog.

- [32] Seyed Iman Mirzadeh, Keivan Alizadeh-Vahid, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. Relu strikes back: Exploiting activation sparsity in large language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- [33] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. OLMoE: Open mixture-of-experts language models, 2024.
- [34] NVIDIA. NCCL: NVIDIA collective communications library. <https://github.com/NVIDIA/ncc1>, 2015. Software library.
- [35] NVIDIA. CUDA graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs>, 2019. CUDA C++ Programming Guide.
- [36] OpenAI. gpt-oss-120b and gpt-oss-20b model card, 2025.
- [37] Ashwinee Panda, Vatsal Sharan, Arturo Marroquin, David Brandfonbrener, Sham Kakade, and Tom Goldstein. Dense backpropagation improves training for sparse mixture-of-experts, 2025.
- [38] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.
- [39] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale, 2019.
- [40] Noam Shazeer. GLU variants improve transformer, 2020.
- [41] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.
- [42] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2014.
- [43] Filip Szatkowski, Patryk Będkowski, Alessio Devoto, Jan Dubiński, Pasquale Minervini, Mikołaj Piórczyński, Simone Scardapane, and Bartosz Wójcik. Universal properties of activation sparsity in modern large language models, 2025.
- [44] Philippe Tillet. *Triton Language API Documentation*, 2020. Accessed: 2026-05-06.
- [45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
- [46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

- [47] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.
- [48] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2024.
- [49] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, 2019. Association for Computational Linguistics.
- [50] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. OPT: Open pre-trained transformer language models, 2022.
- [51] Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. MoEification: Transformer feed-forward layers are mixtures of experts, 2022.
- [52] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianghao Liang, Liangchen Luo, Guandao Yang, Zhangyang Wang, Jian Tang, and Zhangyang Wang. H₂O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [53] Chenggang Zhao, Shangyan Zhou, Liye Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. DeepEP: An efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025. Software library.
- [54] Xu Zhong, Xuefei Ning, Lianghao Guo, Tianchen Zhao, Enshu Liu, Liqiang He, Yi Cai, Kaveh Shamsi, Xuan Tang, Shuaiqi Wang, Yuhao Zhu, Guohao Dai, Huazhong Yang, and Yu Wang. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference, 2024.
- [55] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Quoc V. Le, James Laudon, and Roy Frostig. Mixture-of-experts with expert choice routing, 2022.
- [56] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models, 2022.

A Licenses of the assets used

Table 4: Licenses of the assets used in the paper.

Asset	Type	License	URL
Granite-1B-A400M	Model	Apache 2.0	https://huggingface.co/ibm-granite/granite-3.0-1b-a400m-base
OLMoE-1B-7B	Model	Apache 2.0	https://huggingface.co/allenai/OLMoE-1B-7B-0924
DeepSeek-V2-Lite	Model	DeepSeek Model License	https://huggingface.co/deepseek-ai/DeepSeek-V2-Lite
GPT-OSS-20B	Model	Apache 2.0	https://huggingface.co/openai/gpt-oss-20b
Qwen3.5-35B-A3B	Model	Apache 2.0	https://huggingface.co/Qwen/Qwen3.5-35B-A3B
Qwen3.5-122B-A10B	Model	Apache 2.0	https://huggingface.co/Qwen/Qwen3.5-122B-A10B
Qwen3.5-397B-A17B	Model	Apache 2.0	https://huggingface.co/Qwen/Qwen3.5-397B-A17B
Llama-4-Maverick	Model	Llama 4 Community License	https://huggingface.co/meta-llama/Llama-4-Maverick-17B-128E
ARC-Challenge	Benchmark	Apache 2.0	https://huggingface.co/datasets/allenai/ai2_arc
ARC-Easy	Benchmark	Apache 2.0	https://huggingface.co/datasets/allenai/ai2_arc
HellaSwag	Benchmark	MIT	https://huggingface.co/datasets/Rowan/hellaswag
Winogrande	Benchmark	Apache 2.0	https://huggingface.co/datasets/allenai/winogrande
TruthfulQA	Benchmark	Apache 2.0	https://huggingface.co/datasets/truthful_qa
WikiText-2	Dataset	CC BY-SA 4.0	https://huggingface.co/datasets/Salesforce/wikitext
vLLM	Framework	Apache 2.0	https://github.com/vllm-project/vllm/blob/main/LICENSE
lm-evaluation-harness	Framework	MIT	https://github.com/EleutherAI/lm-evaluation-harness/blob/main/LICENSE

B Intra-expert Activation Sparse MoE Execution Source Code

Listing 1: Full source code of sparse MoE execution kernel.

```

1  """Gate-sparse MoE kernel integration for vLLM.
2
3  Import inside vllm/model_executor/layers/fused_moe/layer.py and inject
4  sparse_moe_forward() at the forward_native() function of the FusedMoE class
5  in layer.py for sparse execution.
6
7  Router computation (softmax + top-k) is performed by vLLM standard router
8  (DefaultMoERunner.router.select_experts) before this module is called;
9  topk_weights and topk_ids are passed in directly.
10
11  """
12
13  from __future__ import annotations
14
15  import functools
16  import logging
17  import os
18  import torch
19  import triton
20
21  from vllm import _custom_ops as ops
22  from vllm.model_executor.layers.fused_moe.fused_moe import (
23      invoke_fused_moe_triton_kernel,
24      try_get_optimal_moe_config,
25      _get_config_dtype_str,
26  )
27
28  logger = logging.getLogger(__name__)
29
30  # -----
31  # One-shot sparsity diagnostic
32  # -----
33  _printed_sparse_map: set = set()
34
35
36  def _maybe_print_sparse_map(
37      layer_id: int,
38      sigma: float,
39      gate_raw_flat: "torch.Tensor", # [T*K, N] post-gate-GEMM, pre-threshold
40      T: int,
41      K: int,
42      N: int,
43  ) -> None:

```

```

44     """Print mask[0] for token-0 expert-0 once per (layer_id, gate_cutoff)
    pair."""
45     key = (layer_id, round(sigma, 6))
46     if key in _printed_sparse_map:
47         return
48     g = gate_raw_flat[0].float().cpu() # token-0, expert-0, shape [N
    ]
49     mask = (g >= sigma).to(torch.int8)
50     n_active = int(mask.sum().item())
51     # Heuristic: skip one-shot when every channel passes a strictly positive
    cutoff
52     # (often uninteresting for diagnostics; old z-score path also used this
    for warmup).
53     if n_active == N and sigma > 0.0:
54         return
55     _printed_sparse_map.add(key)
56     frac = n_active / N
57     print(
58         f"[sparse_moe] layer={layer_id} sigma={sigma:.2f} "
59         f"mask[0]: {n_active}/{N} active frac={frac:.4f} "
60         f"{mask.tolist()}",
61         flush=True,
62     )
63
64     _CAPTURE_LAYER_ID: int | None = (
65         int(os.environ["SPARSE_MOE_CAPTURE_LAYER"])
66         if "SPARSE_MOE_CAPTURE_LAYER" in os.environ else None
67     )
68     _CAPTURE_T: int = int(os.environ.get("SPARSE_MOE_CAPTURE_T", "32"))
69     _CAPTURE_PATH: str = os.environ.get("SPARSE_MOE_CAPTURE_PATH", "")
70     _capture_done: bool = False
71
72     try:
73         import triton.language as tl
74         _SPARSE_MOE_AVAILABLE = True
75     except Exception as _e:
76         _SPARSE_MOE_AVAILABLE = False
77         logger.warning(
78             "Gate-sparse MoE kernel not available: %s. "
79             "Falling back to dense FusedMoE.", _e,
80         )
81
82     # Maximum token count for which the sparse kernel is engaged.
83     # Must match _SPARSE_MAX_T in layer.py.
84     _SPARSE_MAX_T: int = 384
85
86     # -----
87     # Utility functions
88     # -----
89
90     def _next_pow2(n: int) -> int:
91         return 1 << (n - 1).bit_length()
92
93
94     def _estimate_k_total(N: int, K: int, block_k: int = 32) -> int:
95         k = K * N
96         return ((k + block_k - 1) // block_k) * block_k
97
98
99     _SM_COUNT: int | None = None
100
101
102     def _sm_count() -> int:
103         global _SM_COUNT
104         if _SM_COUNT is None:
105             _SM_COUNT = torch.cuda.get_device_properties(
106                 torch.cuda.current_device()
107             ).multi_processor_count

```

```

108     return _SM_COUNT
109
110
111 def k_tiles_for_active_rows(
112     _total_active: torch.Tensor,
113     k_total_pad: int,
114     block_m: int,
115 ) -> int:
116     """Second launch dimension for fused sparse MoE kernels (grid ``(T,
117     K_TILES)``)."""
118     k_cap = triton.cdiv(k_total_pad, block_m)
119     return max(1, k_cap)
120
121 def _get_moe_config(w_gate, w2_shape, k_experts, dtype, T):
122     config_dtype = _get_config_dtype_str(
123         use_fp8_w8a8=False, use_int8_w8a16=False,
124         use_int4_w4a16=False, ocp_mx_scheme=None, dtype=dtype,
125     )
126     return functools.partial(
127         try_get_optimal_moe_config, w_gate.size(), w2_shape,
128         k_experts, config_dtype, block_shape=None,
129     )(T)
130
131
132 # -----
133 # Triton kernels
134 # -----
135
136 if _SPARSE_MOE_AVAILABLE:
137     @triton.jit
138     def sparse_moe_threshold_kernel(
139         GATE_RAW_ptr,
140         TOPK_IDS_ptr,
141         FLAT_ROWS_ptr,
142         ACTIVE_COUNT_ptr,
143         TOTAL_ACTIVE_ptr,
144         gate_cutoff,
145         N: tl.constexpr,
146         K_EXPERTS: tl.constexpr,
147         K_TOTAL_PAD: tl.constexpr,
148         BLOCK_TILE: tl.constexpr,
149     ):
150         pid_t = tl.program_id(0)
151         flat_offset = 0
152
153         for e in tl.static_range(K_EXPERTS):
154             te = pid_t * K_EXPERTS + e
155             gate_base = te * N
156
157             expert_id = tl.load(TOPK_IDS_ptr + pid_t * K_EXPERTS + e)
158             flat_base = expert_id * N
159
160             running = 0
161             for ts in tl.range(0, N, BLOCK_TILE):
162                 offs = ts + tl.arange(0, BLOCK_TILE)
163                 mask = offs < N
164                 g = tl.load(GATE_RAW_ptr + gate_base + offs,
165                             mask=mask, other=0.0).to(tl.float32)
166
167                 active = (g >= gate_cutoff) & mask
168                 slot = tl.cumsum(active.to(tl.int32))
169                 count = tl.sum(active.to(tl.int32))
170
171                 wpos = flat_offset + running + slot - 1
172                 wmask = active & (wpos < K_TOTAL_PAD)
173
174                 tl.store(FLAT_ROWS_ptr + pid_t * K_TOTAL_PAD + wpos,

```

```

175         (flat_base + offs).to(tl.int32), mask=wmask)
176
177         running = running + count
178
179         actual = tl.where(running > K_TOTAL_PAD - flat_offset,
180                          K_TOTAL_PAD - flat_offset, running)
181         tl.store(ACTIVE_COUNT_ptr + te, actual)
182         flat_offset = flat_offset + actual
183
184         tl.store(TOTAL_ACTIVE_ptr + pid_t, flat_offset)
185
186 @triton.jit
187 def sparse_moe_fused_updown_kernel(
188     X_ptr,
189     W_UP_ptr,
190     W_DOWN_ptr,
191     GATE_RAW_ptr,
192     TOPK_IDS_ptr,
193     TOPK_WEIGHTS_ptr,
194     FLAT_ROWS_ptr,
195     TOTAL_ACTIVE_ptr,
196     PARTIAL_ptr,
197     SKIP_STATS_ptr,
198     D: tl.constexpr,
199     N: tl.constexpr,
200     K_EXPERTS: tl.constexpr,
201     BLOCK_KE: tl.constexpr,
202     K_TOTAL_PAD: tl.constexpr,
203     K_TILES_GRID: tl.constexpr,
204     BLOCK_M: tl.constexpr,
205     BLOCK_K: tl.constexpr,
206     BLOCK_N: tl.constexpr,
207     DTYPE: tl.constexpr,
208     RECORD_SKIPS: tl.constexpr,
209     stride_xT, stride_xD,
210     stride_wuE, stride_wuN, stride_wuD,
211     stride_wdE, stride_wdN, stride_wdD,
212     stride_pT, stride_pM,
213 ):
214     pid_t = tl.program_id(0)
215     pid_m = tl.program_id(1)
216
217     total_active = tl.load(TOTAL_ACTIVE_ptr + pid_t)
218     tile_start = pid_m * BLOCK_M
219     if tile_start >= total_active:
220         partial_base = PARTIAL_ptr + pid_t * stride_pT + pid_m *
stride_pM
221         for z in tl.range(0, D, BLOCK_N):
222             zoffs = z + tl.arange(0, BLOCK_N)
223             zmask = zoffs < D
224             tl.store(
225                 partial_base + zoffs,
226                 tl.zeros((BLOCK_N,), dtype=tl.float32),
227                 mask=zmask,
228             )
229             if RECORD_SKIPS:
230                 lin_skip = pid_t * K_TILES_GRID + pid_m
231                 tl.store(SKIP_STATS_ptr + lin_skip, 1)
232             return
233
234     if RECORD_SKIPS:
235         lin_full = pid_t * K_TILES_GRID + pid_m
236         tl.store(SKIP_STATS_ptr + lin_full, 0)
237
238     offs_m = tile_start + tl.arange(0, BLOCK_M)
239     mask_m = offs_m < total_active
240
241     fr = tl.load(FLAT_ROWS_ptr + pid_t * K_TOTAL_PAD + offs_m,

```

```

242         mask=mask_m, other=0)
243
244     expert_id_m = fr // N
245     n_m         = fr % N
246
247     h_acc = tl.zeros((BLOCK_M,), dtype=tl.float32)
248     for k_start in tl.range(0, D, BLOCK_K):
249         offs_k = k_start + tl.arange(0, BLOCK_K)
250         mask_k = offs_k < D
251         x_blk = tl.load(
252             X_ptr + pid_t * stride_xT + offs_k * stride_xD,
253             mask=mask_k, other=0.0,
254         )
255         wu = tl.load(
256             W_UP_ptr + expert_id_m[:, None] * stride_wuE
257                     + n_m[:, None] * stride_wuN
258                     + offs_k[None, :] * stride_wuD,
259             mask=mask_m[:, None] & mask_k[None, :], other=0.0,
260         )
261         h_acc += tl.reshape(tl.dot(wu, x_blk[:, None]), (BLOCK_M,))
262
263     k_offs = tl.arange(0, BLOCK_KE)
264     k_valid = k_offs < K_EXPERTS
265     topk_ids_t = tl.load(TOPK_IDS_ptr + pid_t * K_EXPERTS + k_offs,
266                         mask=k_valid, other=-1)
267     topk_w_t = tl.load(TOPK_WEIGHTS_ptr + pid_t * K_EXPERTS + k_offs,
268                       mask=k_valid, other=0.0).to(tl.float32)
269
270     match = (expert_id_m[:, None] == topk_ids_t[None, :])
271     rw = tl.sum(match.to(tl.float32) * topk_w_t[None, :], axis=1)
272     slot_m = tl.sum(match.to(tl.int32) * k_offs[None, :], axis=1)
273
274     gate_val = tl.load(
275         GATE_RAW_ptr + (pid_t * K_EXPERTS + slot_m) * N + n_m,
276         mask=mask_m, other=0.0,
277     ).to(tl.float32)
278     ag = gate_val * tl.sigmoid(gate_val)
279
280     h = rw * ag * h_acc
281
282     h_row = tl.reshape(h.to(DTYPE), (1, BLOCK_M))
283     partial_base = PARTIAL_ptr + pid_t * stride_pT + pid_m * stride_pM
284
285     for n_start in tl.range(0, D, BLOCK_N):
286         offs_n = n_start + tl.arange(0, BLOCK_N)
287         mask_n = offs_n < D
288         wd = tl.load(
289             W_DOWN_ptr + expert_id_m[:, None] * stride_wdE
290                     + n_m[:, None] * stride_wdN
291                     + offs_n[None, :] * stride_wdD,
292             mask=mask_m[:, None] & mask_n[None, :], other=0.0,
293         )
294         contrib = tl.reshape(tl.dot(h_row, wd), (BLOCK_N,))
295         tl.store(partial_base + offs_n, contrib, mask=mask_n)
296
297 @triton.jit
298 def sparse_moe_accumulate_kernel(
299     PARTIAL_ptr,
300     OUT_PTR,
301     K_TILES,
302     D: tl.constexpr,
303     BLOCK_N: tl.constexpr,
304     DTYPE: tl.constexpr,
305     stride_pT, stride_pM,
306     stride_outT,
307 ):
308     pid_t = tl.program_id(0)
309     pid_n = tl.program_id(1)

```

```

310     offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
311     mask_n = offs_n < D
312
313
314     acc = tl.zeros((BLOCK_N,), dtype=tl.float32)
315     for k_tile in tl.range(0, K_TILES):
316         part = tl.load(
317             PARTIAL_ptr + pid_t * stride_pT + k_tile * stride_pM + offs_n
318             ,
319             mask=mask_n, other=0.0,
320             )
321         acc += part
322
323     tl.store(
324         OUT_PTR + pid_t * stride_outT + offs_n,
325         acc.to(DTYPE),
326         mask=mask_n,
327     )
328 # -----
329 # Workspace management
330 # -----
331
332 class MoEWorkspace:
333     """Grow-never-shrink GPU byte buffer."""
334
335     def __init__(self):
336         self._buf: torch.Tensor | None = None
337
338     def ensure(self, nbytes: int, device: torch.device) -> torch.Tensor:
339         if self._buf is None or self._buf.numel() < nbytes:
340             self._buf = torch.empty(nbytes, dtype=torch.uint8, device=device)
341         return self._buf
342
343     def clear(self) -> None:
344         self._buf = None
345
346
347 _hybrid_workspaces: dict[int, MoEWorkspace] = {}
348 _partial_workspaces: dict[int, MoEWorkspace] = {}
349
350 # Per-layer sparse config: (gate_cutoff from moe_sparse_sigma, k_experts).
351 # renormalize is accepted in register_sparse_config for call-site
352 # compatibility
353 # but is not stored vLLM router handles renormalization internally.
354 _layer_sparse_configs: dict[int, tuple[float, int]] = {}
355
356 _w_down_T_cache: dict[int, torch.Tensor] = {}
357
358 _skip_prog_total: int = 0
359 _skip_prog_skipped: int = 0
360
361 _skip_dummy_by_device: dict[torch.device, torch.Tensor] = {}
362
363 def _get_skip_dummy(device: torch.device) -> torch.Tensor:
364     t = _skip_dummy_by_device.get(device)
365     if t is None:
366         t = torch.zeros(1, dtype=torch.int32, device=device)
367         _skip_dummy_by_device[device] = t
368     return t
369
370
371 def reset_fused_skip_program_stats() -> None:
372     global _skip_prog_total, _skip_prog_skipped
373     _skip_prog_total = 0
374     _skip_prog_skipped = 0
375

```

```

376
377 def fused_skip_program_stats() -> dict[str, float]:
378     tot = _skip_prog_total
379     sk = _skip_prog_skipped
380     return {
381         "programs_total": float(tot),
382         "programs_skipped": float(sk),
383         "skip_fraction": float(sk / tot) if tot else 0.0,
384     }
385
386
387 def register_sparse_config(
388     layer_id: int, sigma: float, k_experts: int, renormalize: bool,
389 ) -> None:
390     """Pre-register per-layer sparse config so the custom_op can look it up."""
391     _layer_sparse_configs[layer_id] = (sigma, k_experts)
392
393
394 def _get_workspaces(layer_id: int):
395     if layer_id not in _hybrid_workspaces:
396         _hybrid_workspaces[layer_id] = MoEWorkspace()
397         _partial_workspaces[layer_id] = MoEWorkspace()
398     return _hybrid_workspaces[layer_id], _partial_workspaces[layer_id]
399
400
401 def clear_workspaces() -> None:
402     for ws in _hybrid_workspaces.values():
403         ws.clear()
404     for ws in _partial_workspaces.values():
405         ws.clear()
406     _hybrid_workspaces.clear()
407     _partial_workspaces.clear()
408     _layer_sparse_configs.clear()
409     _w_down_T_cache.clear()
410     _skip_dummy_by_device.clear()
411
412
413 # -----
414 # Forward implementation
415 # -----
416
417 def _sparse_moe_forward_impl(
418     hidden_states: torch.Tensor,
419     topk_weights: torch.Tensor, # [T, K] fp32 from vLLM router
420     topk_ids: torch.Tensor, # [T, K] int32 from vLLM router
421     w13_weight: torch.Tensor,
422     w2_weight: torch.Tensor,
423     layer_id: int,
424 ) -> torch.Tensor:
425     sigma, k_experts = _layer_sparse_configs[layer_id]
426     T, D = hidden_states.shape
427     E, N2, _ = w13_weight.shape
428     N = N2 // 2
429     K = k_experts
430     TK = T * K
431
432     w_gate = w13_weight[:, :N, :]
433     w_up = w13_weight[:, N:, :]
434     w_down_T = _w_down_T_cache.get(layer_id)
435     if w_down_T is None:
436         w_down_T = w2_weight.permute(0, 2, 1).contiguous() # [E, N, D]
437         _w_down_T_cache[layer_id] = w_down_T
438         # Only offload when sparse-only mode is guaranteed (no dense fallback
439         # possible).
440         # if os.environ.get("SPARSE_MOE_OFFLOAD_W2", "0") == "1":
441         #     w2_weight.data = w2_weight.cpu() # free GPU copy;
442         w_down_T is the live reference

```

```

441 BLOCK_TILE = min(1024, _next_pow2(N))
442 K_TOTAL_PAD = _estimate_k_total(N, K, sigma)
443
444
445 device = hidden_states.device
446 dtype = hidden_states.dtype
447 elem = hidden_states.element_size()
448 tl_dtype = tl.bfloat16 if dtype == torch.bfloat16 else tl.float16
449
450 config = _get_moe_config(w_gate, w2_weight.size(), K, dtype, T)
451 ALIGN_BS = config["BLOCK_SIZE_M"]
452
453 ws, partial_ws = _get_workspaces(layer_id)
454
455 _align = 256
456 _T_max = max(T, _SPARSE_MAX_T)
457
458 def _al(n): return (n + _align - 1) // _align * _align
459
460 _max_tok_pad_ws = _T_max * K + E * (ALIGN_BS - 1)
461 _max_m_blks_ws = triton.cdiv(_max_tok_pad_ws, ALIGN_BS)
462
463 R1 = _al(_T_max * K * 2) # topk_weights_ws fp16
464 R2 = _al(_T_max * K * 4) # topk_ids_ws int32
465 R3 = _al(_T_max * K * N * elem) # gate_raw
466 R4 = _al(_T_max * K_TOTAL_PAD * 4) # flat_rows int32
467 R6 = _al(_T_max * K * 4) # active_count int32
468 R6b = _al(_T_max * 4) # total_active int32
469 R_ST = _al(_max_tok_pad_ws * 4) # sorted_token_ids int32
470 R_EI = _al(_max_m_blks_ws * 4) # expert_ids int32
471 R_NTP = _al(4) # num_tokens_pp int32
472 R_OUT = _al(_T_max * D * elem) # out
473
474 total = R1 + R2 + R3 + R4 + R6 + R6b + R_ST + R_EI + R_NTP + R_OUT
475 buf = ws.ensure(total, device)
476
477 off = 0
478 o1 = off; off += R1
479 o2 = off; off += R2
480 o3 = off; off += R3
481 o4 = off; off += R4
482 o6 = off; off += R6
483 o6b = off; off += R6b
484 o_st = off; off += R_ST
485 o_ei = off; off += R_EI
486 o_ntp = off; off += R_NTP
487 o_out = off; off += R_OUT
488
489 topk_weights_ws = buf[o1 : o1 + T*K*2].view(torch.float16).reshape(
490 , K)
491 topk_ids_ws = buf[o2 : o2 + T*K*4].view(torch.int32).reshape(
492 , K)
493 gate_raw = buf[o3 : o3 + T*K*N*elem].view(dtype).reshape(T
494 , K, N)
495 flat_rows = buf[o4 : o4 + T*K_TOTAL_PAD*4].view(torch.int32).
496 reshape(T, K_TOTAL_PAD)
497 active_count = buf[o6 : o6 + TK*4].view(torch.int32).reshape(
498 TK)
499 total_active = buf[o6b: o6b + T*4].view(torch.int32).reshape(
500 T)
501 _max_tok_pad = TK + E * (ALIGN_BS - 1)
502 _max_m_blks = triton.cdiv(_max_tok_pad, ALIGN_BS)
503 sorted_token_ids = buf[o_st : o_st + _max_tok_pad * 4].view(torch.int32
504 )
505 expert_ids = buf[o_ei : o_ei + _max_m_blks * 4].view(torch.int32
506 )
507 num_tokens_pp = buf[o_ntp : o_ntp + 4].view(torch.int32
508 )

```

```

500 out = buf[o_out : o_out + T*D*elem ].view(dtype
501 ).reshape(T, D)
502
503 # Copy vLLM router outputs into fp16 workspace slots expected by Triton
504 kernels
505 topk_weights_ws.copy_(topk_weights.to(torch.float16))
506 topk_ids_ws.copy_(topk_ids.to(torch.int32))
507
508 # S2b: expert permutation
509 ops.moe_align_block_size(
510     topk_ids_ws, E, ALIGN_BS,
511     sorted_token_ids, expert_ids, num_tokens_pp, None,
512 )
513
514 # S3a: gate GEMM
515 invoke_fused_moe_triton_kernel(
516     hidden_states, w_gate, gate_raw,
517     None, None, None,
518     sorted_token_ids, expert_ids, num_tokens_pp,
519     False, K, config, tl_dtype,
520     False, False, False, False, False, None, None,
521 )
522 gate_raw_flat = gate_raw.reshape(TK, N)
523
524 # S3b: absolute gate cutoff + compact (`sigma` is `moe_sparse_sigma`)
525 sparse_moe_threshold_kernel[(T,)](
526     gate_raw_flat,
527     topk_ids_ws,
528     flat_rows,
529     active_count, total_active,
530     sigma, # gate_cutoff in kernel
531     N=N, K_EXPERTS=K, K_TOTAL_PAD=K_TOTAL_PAD,
532     BLOCK_TILE=BLOCK_TILE,
533 )
534
535 _maybe_print_sparse_map(layer_id, sigma, gate_raw_flat, T, K, N)
536
537 # S4a/b: fused up+down + accumulate
538 _T_ref = _SPARSE_MAX_T
539 _sm = _sm_count()
540 _bm = min(128, max(32, _next_pow2(_T_ref * K_TOTAL_PAD // (4 * max(_sm,
541 1)))))
542 while _T_ref * triton.cdiv(K_TOTAL_PAD, _bm) < _sm and _bm > 16:
543     _bm //= 2
544 _bn = 64
545
546 K_TILES = k_tiles_for_active_rows(total_active, K_TOTAL_PAD, _bm)
547 _part_buf = partial_ws.ensure(_T_max * K_TILES * D * 4, device)
548 partial = _part_buf[: T * K_TILES * D * 4].view(torch.float32).reshape(T,
549 K_TILES, D)
550
551 _record_skips = os.environ.get("VLLM_SPARSE_MOE_RECORD_SKIP_STATS", "")
552 == "1"
553 if _record_skips:
554     skip_program_buf = torch.zeros(T * K_TILES, dtype=torch.int32, device
555 =device)
556     skip_stats_ptr = skip_program_buf
557 else:
558     skip_stats_ptr = _get_skip_dummy(device)
559
560 sparse_moe_fused_updown_kernel[(T, K_TILES)](
561     hidden_states, w_up, w_down_T,
562     gate_raw_flat, topk_ids_ws, topk_weights_ws,
563     flat_rows, total_active,
564     partial,
565     skip_stats_ptr,
566     D=D, N=N, K_EXPERTS=K, BLOCK_KE=_next_pow2(K), K_TOTAL_PAD=
567 K_TOTAL_PAD,

```

```

561         K_TILES_GRID=K_TILES,
562         BLOCK_M=_bm, BLOCK_K=64, BLOCK_N=_bn,
563         DTYPE=tl_dtype,
564         RECORD_SKIPS=_record_skips,
565         stride_xT=hidden_states.stride(0), stride_xD=hidden_states.stride(1)
566     ,
567     stride_wuE=w_up.stride(0), stride_wuN=w_up.stride(1),
568     stride_wuD=w_up.stride(2),
569     stride_wdE=w_down_T.stride(0), stride_wdN=w_down_T.stride(1),
570     stride_wdD=w_down_T.stride(2),
571     stride_pT=partial.stride(0), stride_pM=partial.stride(1),
572 )
573
574 if _record_skips:
575     torch.cuda.synchronize()
576     global _skip_prog_total, _skip_prog_skipped
577     n_prog = T * K_TILES
578     n_skip = int(skip_program_buf.sum().item())
579     _skip_prog_total += n_prog
580     _skip_prog_skipped += n_skip
581
582 sparse_moe_accumulate_kernel[(T, triton.cdiv(D, _bn))](
583     partial, out,
584     K_TILES=K_TILES,
585     D=D, BLOCK_N=_bn,
586     DTYPE=tl_dtype,
587     stride_pT=partial.stride(0), stride_pM=partial.stride(1),
588     stride_outT=out.stride(0),
589 )
590
591 # One-shot capture (no-op unless SPARSE_MOE_CAPTURE_LAYER is set)
592 global _capture_done
593 if (
594     _CAPTURE_LAYER_ID is not None
595     and not _capture_done
596     and layer_id == _CAPTURE_LAYER_ID
597     and T == _CAPTURE_T
598 ):
599     import pathlib
600     save_path = _CAPTURE_PATH or f"sparse_moe_capture_layer{layer_id}.pt"
601     pathlib.Path(save_path).parent.mkdir(parents=True, exist_ok=True)
602     torch.save(
603         {
604             "hidden_states": hidden_states.detach().cpu().clone(),
605             "topk_weights": topk_weights.detach().cpu().clone(),
606             "topk_ids": topk_ids.detach().cpu().clone(),
607             "w13_weight": w13_weight.detach().cpu().clone(),
608             "w2_weight": w2_weight.detach().cpu().clone(),
609             "layer_id": layer_id,
610             "T": T, "D": D, "E": E, "N": N, "K": K,
611         },
612         save_path,
613     )
614     logger.info(
615         "[sparse_moe] Captured layer_id=%d T=%d D=%d E=%d N=%d K=%d: %s",
616         layer_id, T, D, E, N, K, save_path,
617     )
618     _capture_done = True
619
620 return out
621
622 # -----
623 # torch.library.custom_op registration
624 # -----
625
626 if _SPARSE_MOE_AVAILABLE:
627     @torch.library.custom_op(

```

```

626         "vllm_sparse_moe::hybrid_forward",
627         mutates_args=(),
628     )
629     def sparse_moe_forward(
630         hidden_states: torch.Tensor,
631         topk_weights: torch.Tensor,
632         topk_ids: torch.Tensor,
633         w13_weight: torch.Tensor,
634         w2_weight: torch.Tensor,
635         layer_id: int,
636     ) -> torch.Tensor:
637         return _sparse_moe_forward_impl(
638             hidden_states, topk_weights, topk_ids,
639             w13_weight, w2_weight, layer_id,
640         )
641
642     @sparse_moe_forward.register_fake
643     def _sparse_moe_forward_fake(
644         hidden_states, topk_weights, topk_ids,
645         w13_weight, w2_weight, layer_id,
646     ):
647         T, D = hidden_states.shape
648         return hidden_states.new_empty(T, D)
649
650 else:
651     def sparse_moe_forward(*args, **kwargs) -> torch.Tensor: # type: ignore[
652         misc]
653         raise RuntimeError("Gate-sparse MoE kernel not available.")

```